

**LAPORAN UTS
COMMUNICATION PROTOCOLS**

CASE 7 PROTOBUF/gRPC



Dosen Mata Kuliah: Haikal Shiddiq, S.Kom, M.T.

Disusun Oleh:

Ahmad Muwaffaq

(25120500007)

**PROGRAM STUDI SAINS DATA
FAKULTAS ILMU KOMPUTER
UNIVERSITAS CAKRAWALA
2025**

RINGKASAN

1. Tujuan Protokol

Remote Procedure Call (RPC) bertujuan untuk memperbolehkan sebuah program komputer mengeksekusi fungsi atau bagian dari program lain pada komputer atau server yang berbeda secara lintas jaringan. RPC menyederhanakan detail dari jaringan yang kompleks sehingga pengembang dapat menulis kode untuk berkomunikasi dengan sistem jarak jauh semudah memanggil fungsi lokal (di tempat/program yang sama).

2. Skenario Bisnis

Dalam bisnis, RPC mendukung kecepatan yang tinggi, arsitektur terdistribusi (*microservices*) dengan menangani rute jaringan serta serialisasinya yang secara drastis dapat menurunkan *latency* dan hambatan pada saat integrasi.

Protokol PRC ini bekerja dengan sangat baik untuk bisnis *real-time payment processing* dan *high-frequency algorithmic trading*. Seperti pada memproses pembayaran secara *real-time*, sistem diharuskan langsung mengkomunikasikan secara aman pembayaran dari pelanggan kepada *payment gateway* untuk memproses transaksi tersebut; pada skenario ini, RPC sangat membantu karena permintaan tersebut dikirim melalui HTTP/2 yang pada dasarnya cepat dan format yang dikirim berupa binari.

DESAIN REQUEST

1. Method `/InferenceService/HealthCheck`

a. Body

```
```\n{\n```\n
```

#### b. Expected response

```
```\njson\n{\n  "status": "SERVING",\n  "transport": "gRPC over HTTP/2"\n}\n```\n
```

2. Method `/InferenceService/Predict`

a. Skenario 1

i. Body

```
```\njson\n{\n  "customerId": "C-1024",\n  "features": {\n    "avgOrderValue": 250000,\n    "monthlyOrders": 7\n  }\n}\n```\n
```

##### ii. Expected Response

```
```\njson\n{\n  "customer_id": "C-1024",\n  "score": 0.59,\n  "label": "standard",\n  "model_version": "integrated-http2-v1"\n}\n```\n
```

b. Skenario 2

i. Body

```
```\njson\n{\n  "customerId": "C-1025",\n  "features": {\n    "avgOrderValue": 125000000,\n    "monthlyOrders": 100\n  }\n}\n```\n
```

```
 }
 }
 ...
```

ii. Expected Response

```
```json  
{  
  "customer_id": "C-1025",  
  "score": 1,  
  "label": "high_value",  
  "model_version": "integrated-http2-v1"  
}  
```
```

3. Method **/InferenceService/BatchPredict**

a. Body

```
```json  
{  
  "items": [  
    {  
      "customerId": "C-1001",  
      "features": {  
        "avgOrderValue": 150000,  
        "monthlyOrders": 3  
      }  
    },  
    {  
      "customerId": "C-1002",  
      "features": {  
        "avgOrderValue": 420000,  
        "monthlyOrders": 9  
      }  
    }  
  ]  
}  
}EV  
```
```

## EVIDENCE

Proto Contract

GET .protoGET Services

RPC Method Lab

RPC Method

HealthCheck

{}

POST Concept MockPOST Dynamic RPC Call

POST Real gRPC HTTP/2 Call

Quick Invoke

Predict ConceptPredict HTTP/2

BatchPredict ConceptBatchPredict HTTP/2

HealthCheck ConceptHealthCheck HTTP/2

Jika HTTP/2 tidak aktif karena port `50051` bentrok, output tetap menampilkan `fallbackConceptResponse` agar demo web tidak menggantung.

Output

POST /api/grpc/http2-call  
HTTP 200 OK

```
{
 "ok": true,
 "realGrpc": true,
 "triggerEndpoint": "POST /api/grpc/http2-call",
 "method": "HealthCheck",
 "target": "0.0.0.0:50051",
 "transport": "gRPC over HTTP/2",
 "request": {},
 "response": {
 "status": "SERVING",
 "transport": "gRPC over HTTP/2"
 },
 "wireshark": {
 "captureFilter": "tcp port 50051",
 "displayFilter": "http2",
 "decodeAs": "TCP port 50051 -> HTTP2"
 },
 "note": "HTTP endpoint ini hanya trigger. Traffic yang perlu di-capture adalah gRPC HTTP/2 pada port 50051."
}
```

### 1.1 Evidence Pemanggilan Method `/InferenceService/HealthCheck`

Pada pemanggilan *method* `/InferenceService/HealthCheck`, *RPC client* memanggil sebuah fungsi yang berada di program/server lain bernama `HealthCheck` dengan *request body* kosong dan akan mengembalikan sebuah status yang menandakan apakah layanan sedang aktif atau tidak.

Proto Contract

GET .proto

GET Services

RPC Method Lab

RPC Method

Predict

```
{
 "customerId": "C-1024",
 "features": {
 "avgOrderValue": 250000,
 "monthlyOrders": 7
 }
}
```

POST Concept Mock

POST Dynamic RPC Call

POST Real gRPC HTTP/2 Call

Quick Invoke

Predict Concept

Predict HTTP/2

BatchPredict Concept

BatchPredict HTTP/2

HealthCheck Concept

HealthCheck HTTP/2

Jika HTTP/2 tidak aktif karena port `50051` bentrok, output tetap menampilkan `fallbackConceptResponse` agar demo web tidak menggantung.

Output

```
POST /api/grpc/http2-call
HTTP 200 OK

{
 "ok": true,
 "realGrpc": true,
 "triggerEndpoint": "POST /api/grpc/http2-call",
 "method": "Predict",
 "target": "0.0.0.0:50051",
 "transport": "gRPC over HTTP/2",
 "request": {
 "customer_id": "C-1024",
 "features": {
 "avg_order_value": 250000,
 "monthly_orders": 7
 }
 },
 "response": {
 "customer_id": "C-1024",
 "score": 0.59,
 "label": "standard",
 "model_version": "integrated-http2-v1"
 },
 "wireshark": {
 "captureFilter": "tcp port 50051",
 "displayFilter": "http2",
 "decodeAs": "TCP port 50051 -> HTTP2"
 },
 "note": "HTTP endpoint ini hanya trigger. Traffic yang perlu di-capture adalah gRPC HTTP/2 pada port 50051."
}
```

### 1.2 Evidence Pemanggilan Method /InferenceService/Predict

Pada pemanggilan *method* /InferenceService/Predict, *RPC client* harus membawa sebuah *request body* yang berisi *customerId*, *features.avgOrderValue*, *features.monthlyOrders* yang nantinya akan diproses dan server akan mengembalikan dengan format *customerId*, *score*, *label*, dan *model\_version*. Kontrak dari *request body* dan *response* tersebut ditulis dalam file *.proto*.

Atribut *features.avgOrderValue* dan *features.monthlyOrders* akan memengaruhi respon *score* dan *label*.

Proto Contract

GET .proto

GET Services

RPC Method Lab

RPC Method

BatchPredict

```
{
 "items": [
 {
 "customerId": "C-1001",
 "features": {
 "avgOrderValue": 150000,
 "monthlyOrders": 3
 }
 },
 {
 "customerId": "C-1002",
 "features": {
 "avgOrderValue": 420000,
 "monthlyOrders": 9
 }
 }
]
}
```

POST Concept Mock

POST Dynamic RPC Call

POST Real gRPC HTTP/2 Call

Quick Invoke

Predict Concept

Predict HTTP/2

BatchPredict Concept

BatchPredict HTTP/2

HealthCheck Concept

HealthCheck HTTP/2

Jika HTTP/2 tidak aktif karena port `50051` bentrok, output tetap menampilkan `fallbackConceptResponse` agar demo web tidak menggantung.

Output

```
POST /api/grpc/http2-call
HTTP 200 OK

{
 "ok": true,
 "realGrpc": true,
 "triggerEndpoint": "POST /api/grpc/http2-call",
 "method": "Predict",
 "target": "0.0.0.0:50051",
 "transport": "gRPC over HTTP/2",
 "request": {
 "customer_id": "C-1025",
 "features": {
 "avg_order_value": 125000000,
 "monthly_orders": 100
 }
 },
 "response": {
 "customer_id": "C-1025",
 "score": 1,
 "label": "high_value",
 "model_version": "integrated-http-v1"
 },
 "wireshark": {
 "captureFilter": "tcp port 50051",
 "displayFilter": "http2",
 "decodeAs": "TCP port 50051 -> HTTP2"
 },
 "note": "HTTP endpoint ini hanya trigger. Traffic yang perlu di-capture adalah gRPC HTTP/2 pada port 50051."
}
```

### 1.2 Evidence Pemanggilan Method /InferenceService/BatchPredict

Method /InferenceService/BatchPredict dipanggil dengan skema *request body* yang sama dengan *method* /InferenceService/Predict, namun dibungkus oleh *items* yang berupa *list/array* yang berisi *customerId*, *features.avgOrderValue*, dan *features.monthlyOrders*.

## ANALISIS ENCODING

Protobuf adalah sebuah bahasa untuk mendefinisikan kontrak secara ketat, dalam binari, dan teroptimasi untuk *microservices*. Berkas `inference.proto` berfungsi sebagai kontrak agnostik yang bisa digunakan oleh berbagai *tools* dan bahasa.

Bisa dilihat pada berkas tersebut, di bagian paling atas ada definisi daftar *service* apa yang berjalan. Lalu di bawahnya kita bisa mendefinisikan bagaimana bentuk dari *request* dan *response* pada *service* tersebut.

```
syntax = "proto3";

package demo.inference;

service InferenceService {
 rpc Predict (PredictionRequest) returns (PredictionReply) {}
 rpc BatchPredict (BatchPredictionRequest) returns (BatchPredictionReply) {}
 rpc HealthCheck (HealthCheckRequest) returns (HealthCheckReply) {}
}

message PredictionRequest {
 string customer_id = 1;
 FeatureVector features = 2;
}

message FeatureVector {
 double avg_order_value = 1;
 int32 monthly_orders = 2;
}

message PredictionReply {
 string customer_id = 1;
 double score = 2;
 string label = 3;
 string model_version = 4;
}

message BatchPredictionRequest {
 repeated PredictionRequest items = 1;
}

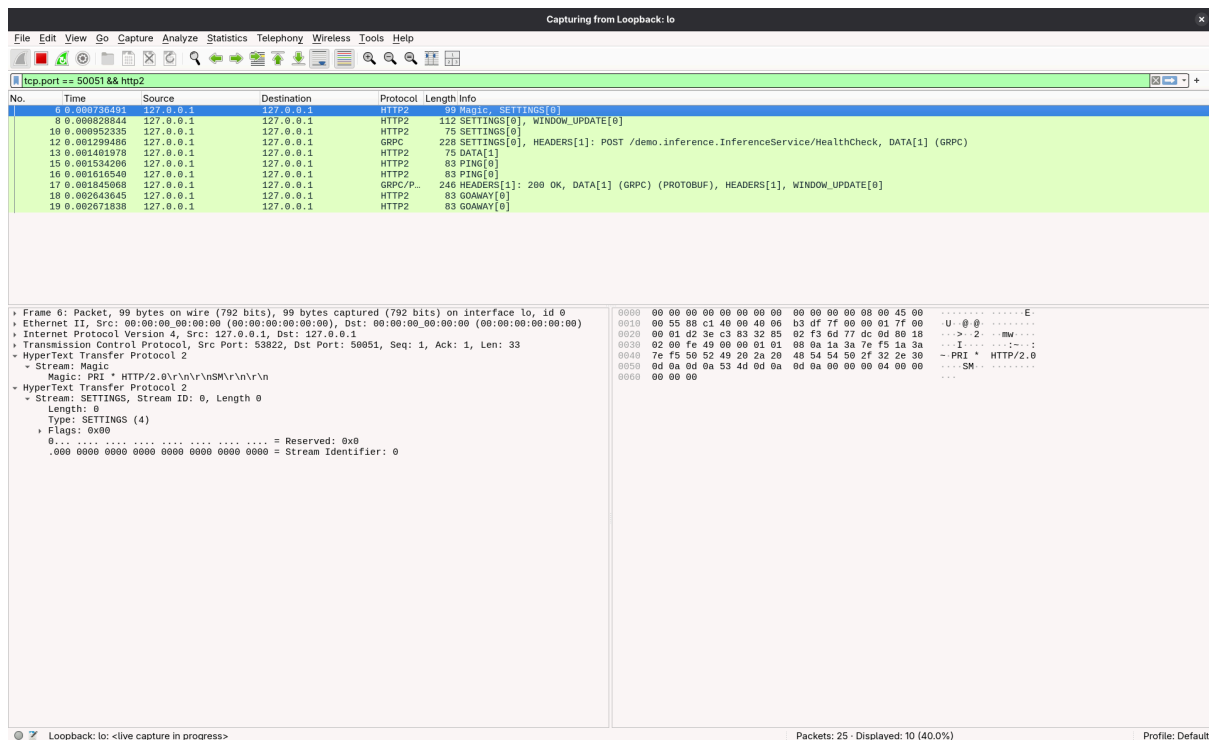
message BatchPredictionReply {
 repeated PredictionReply predictions = 1;
}

message HealthCheckRequest {}

message HealthCheckReply {
 string status = 1
 string transport = 2
}
```

### 1.3 Kontrak Protobuf





#### 1.4 Inspeksi Pemanggilan RPC dengan Wireshark

### 1. Inisialisasi Koneksi & Preface (Frame 6, 8, 10)

Sebelum data aplikasi dapat dikirim, HTTP/2 memerlukan jabat tangan (*handshake*) berupa *connection preface* untuk memastikan kedua belah pihak mendukung protokol ini.

- Frame 6 (Magic, SETTINGS[0]):** Klien mengirimkan string *connection preface* HTTP/2 (`PRI * HTTP/2.0\r\n\r\nSM\r\n\r\n`), seperti yang terlihat pada detail paket di file `image_cf7c7e.png`. Klien juga langsung mengirimkan frame `SETTINGS` pada **Stream 0** (jalur kontrol) untuk mengumumkan parameter koneksinya.
- Frame 8 & 10 (SETTINGS[0], WINDOW\_UPDATE[0]):** Server merespons dengan pengaturannya sendiri (*settings*) dan mengirimkan frame `WINDOW_UPDATE` untuk mengelola ukuran jendela kendali aliran (*flow control window*) di tingkat koneksi.

### 2. Permintaan gRPC / gRPC Request (Frame 12 & 13)

Setelah koneksi HTTP/2 terjalin, klien memulai panggilan gRPC yang sebenarnya.

- Frame 12 (HEADERS[1], DATA[1]):**
  - Stream ID 1:** Klien membuka *stream* baru (**Stream 1**) yang dikhususkan untuk permintaan ini.
  - HEADERS:** gRPC menerjemahkan panggilan RPC menjadi permintaan HTTP/2 `POST`. Jalur (*path*) yang digunakan adalah `/demo.inference.InferenceService/HealthCheck`. Ini menunjukkan bahwa klien sedang memanggil metode `HealthCheck` pada *service* bernama `InferenceService`.

iii. **DATA:** *Payload* aktual yang berisi argumen permintaan (yang diserialisasi menggunakan Protocol Buffers) dikirim langsung setelah *headers* pada *stream* yang sama.

b. **Frame 13 (DATA[1]):** Berisi data tambahan atau sinyal *flow control* yang berkaitan dengan Stream 1.

### 3. Pemeliharaan Koneksi / Keep-Alive (Frame 15 & 16)

a. **PING[0]:** Klien dan server saling bertukar frame *ping/pong* pada Stream 0. Aktivitas ini digunakan untuk mengukur *Round-Trip Time* (RTT) atau sebagai mekanisme *keep-alive* agar koneksi tidak terputus karena dianggap menganggur (*idle*).

### 4. Respons gRPC / gRPC Response (Frame 17)

a. **Frame 17 (HEADERS[1], DATA[1] (PROTOBUF)):** Server memproses permintaan dan mengirimkan balasannya pada *stream* yang sama (**Stream 1**).

i. **HEADERS (Response):** Server mengirimkan status HTTP **200 OK**.

ii. **DATA (PROTOBUF):** Berisi respons Protobuf yang telah diserialisasi dari RPC **HealthCheck** (misalnya status kesehatan *service*).

iii. **Trailing HEADERS:** Dalam gRPC, sekumpulan *headers* kedua (disebut *trailers*) dikirim di bagian paling akhir *stream*. Di sinilah status kode gRPC yang sebenarnya berada (misalnya, **grpc-status: 0**, yang berarti **OK** / Sukses), yang menandakan bahwa operasi RPC telah selesai dengan sukses.

### 5. Pemutusan Koneksi / Teardown (Frame 18 & 19)

a. **GOAWAY[0]:** Kedua belah pihak bertukar frame **GOAWAY** pada Stream 0. Frame ini memberi tahu pihak lawan bahwa tidak akan ada *stream* baru yang dibuat di koneksi ini, sehingga sesi dapat ditutup secara bersih dan aman (*graceful shutdown*).

### 6. Ringkasan Poin Penting

a. **Multiplexing:** Perhatikan bagaimana sinyal kontrol (**SETTINGS**, **PING**, **GOAWAY**) berjalan pada **Stream 0**, sedangkan data aplikasi gRPC yang sebenarnya diisolasi pada **Stream 1**.

b. **Pemetaan RPC-ke-HTTP/2:** Nama *service* dan metode dipetakan langsung menjadi struktur jalur URL dengan format: **/{{Package}}.{{Service}}/{{Method}}**.

## KESIMPULAN

### 1. Temuan Utama

Saya menemukan bagaimana *stream* pada HTTP/2 itu berjalan dengan menginspeksinya di Wireshark. Sehingga saya mengerti bagaimana *stream* tersebut dimulai dan diakhiri.

### 2. Kendala

Saya terkendala untuk menemukan *case error handling* dikarenakan server selalu merespon dengan benar kecuali saya mengirimkan *request body* yang kosong.

### 3. Pembelajaran Protokol

Saya menemukan protokol RPC ini sangat menarik yang dimana kontrak komunikasi yang ditulis itu agnostik dan tidak terikat pada bahasa apa pun, lalu *tooling* yang digunakan untuk menghasilkan kode aplikasi yang *type-safe* juga sangat membantu developer mempercepat pekerjaan.

## DOKUMEN LAIN

- **GitHub:** <https://github.com/mupinnn/communication-protocols-midtest-report>
- **Presentation:** [https://docs.google.com/presentation/d/1gCT\\_mFZS-6A5SQ0tUZqGcoFrXKqltc5HvZ101rZ1a0/edit?usp=sharing](https://docs.google.com/presentation/d/1gCT_mFZS-6A5SQ0tUZqGcoFrXKqltc5HvZ101rZ1a0/edit?usp=sharing)